

使用 BigQuery 圖表偵測詐欺

來源：<https://codelabs.developers.google.com/codelabs/fraud-bigquery-graph?hl=zh-tw>

步驟數：10

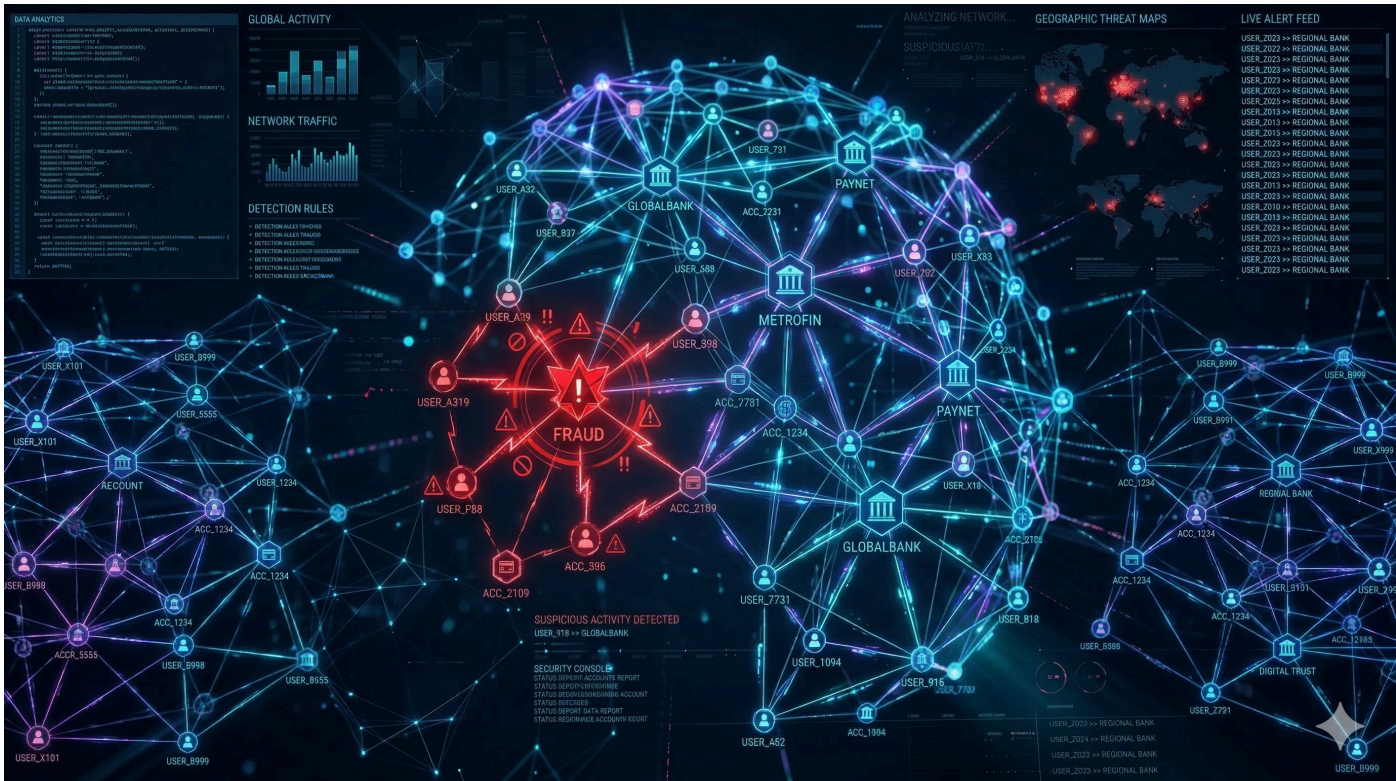
瞭解如何使用 BigQuery Graph 偵測詐欺網路。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 載入資料](#)
4. [4. 建立屬性圖表](#)
5. [5. 分析網路 \(2 個躍點\)](#)
6. [6. 分析網路 \(4 個躍點\)](#)
7. [7. 產生詐欺報表](#)
8. [8. 大規模偵測](#)
9. [9. 清除](#)
10. [10. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介



詐欺活動通常會涉及隱藏的實體連結網路，例如多個帳戶共用相同的電子郵件地址、電話號碼或實際地址。傳統關聯式資料庫可能難以有效率地查詢這些複雜的多跳關係。

BigQuery Graph 可讓您使用圖形資料庫，大規模分析這些網路。您可以在現有的 BigQuery 資料表上定義屬性圖，並使用 Graph Query Language (GQL) 尋找資料中的模式。

圖形網路在詐欺偵測方面的一項常見應用，就是停止與詐欺網路相關聯的寄送地址訂單，或是停止屬於這類網路的付款。

在本程式碼研究室中，您將使用 BigQuery 圖表建構詐欺偵測解決方案。您將從 Cloud Storage 載入資料、建立屬性圖，並使用圖形查詢找出可疑的連結。

課程內容

- 如何建立 BigQuery 資料集並載入資料。
- 如何使用 DDL 定義屬性圖。
- 如何使用 GQL 查詢圖形。
- 如何使用圖表分析偵測詐欺行為。

軟硬體需求

- 已啟用計費功能的 Google Cloud 專案。
- BigQuery 筆記本環境 (BigQuery Studio 或 Colab Enterprise)。

費用

本實驗室會使用計費的 Google Cloud 資源。假設您在完成後刪除資源，預估費用將低於 \$5 美元。

2. 事前準備

選取或建立 Google Cloud 專案

1. 在 Google Cloud 控制台的專案選取器頁面中，選取或建立 Google Cloud 專案。
2. 請確認 Google Cloud 專案已啟用計費功能。瞭解如何[檢查是否已啟用計費功能](#)。

選擇環境

您需要筆記本環境才能執行本實驗室。您可以使用 **BigQuery Studio** 或 **Colab Enterprise**。

1. 前往 Google Cloud 控制台的「BigQuery」**BigQuery**頁面。
2. 您將使用 Python 筆記本執行圖表查詢。

啟動 Cloud Shell

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 驗證：

□□□□

```
gcloud auth list
```

3. 確認專案：

□□□□

```
gcloud config get project
```

4. 視需要設定：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用 API

執行下列指令，啟用必要的 BigQuery API：

```
gcloud services enable bigquery.googleapis.com
```

步驟 3 · 約 5 分鐘

3. 載入資料

在這個步驟中，您將建立 BigQuery 資料集，並從 Cloud Storage 載入範例資料。

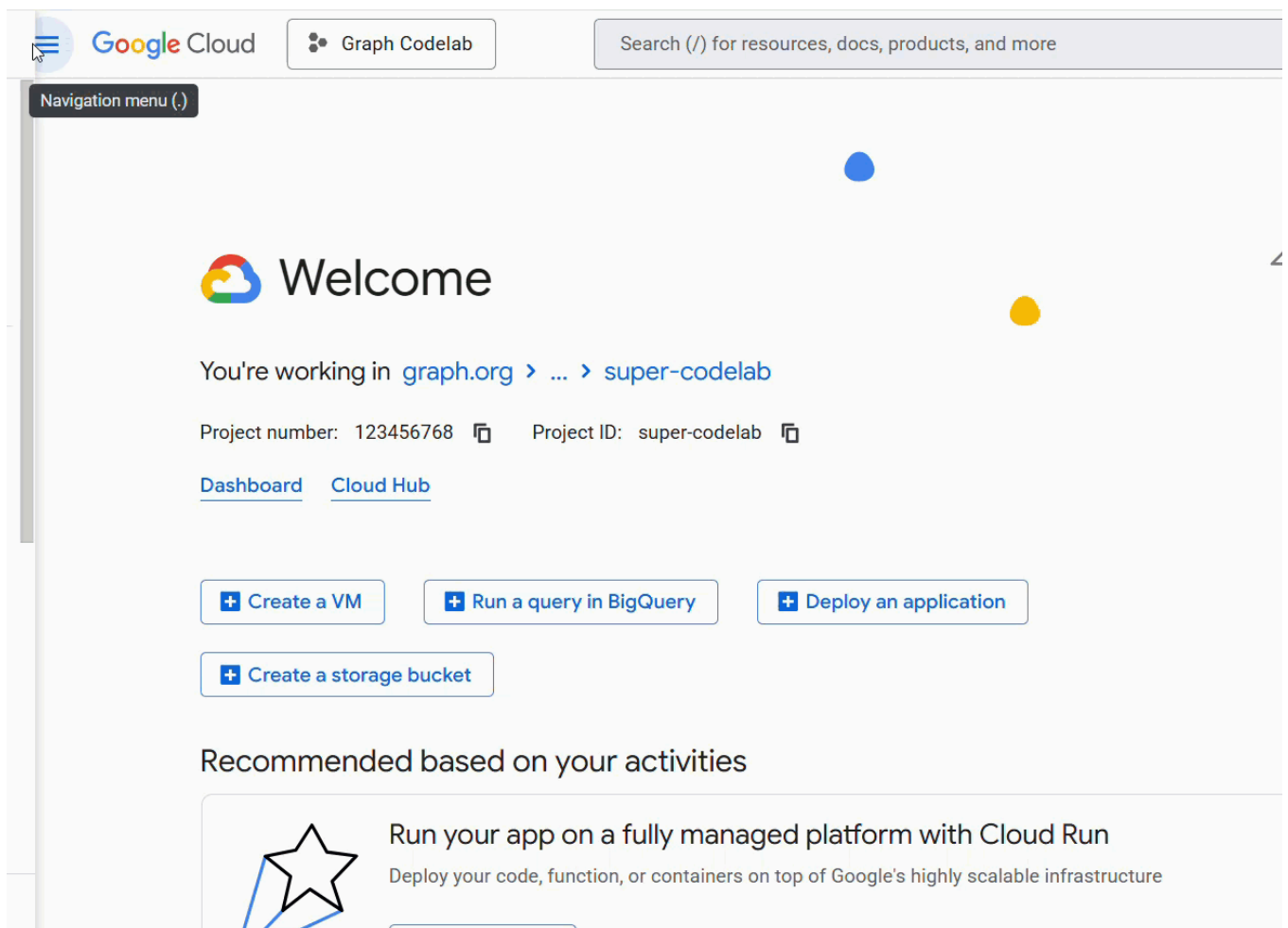
範例資料包含多個 CSV 檔案，代表模擬的零售環境：

- `customers.csv`：客戶帳戶資訊。
- `emails.csv`：電子郵件地址。
- `phones.csv`：電話號碼。
- `addresses.csv`：實際地址。
- `customer_emails.csv`、`customer_phones.csv`、`customer_addresses.csv`：連結表格。
- `orders.csv`：訂單記錄，包括詐欺標記。

建立資料集

建立名為 `fraud_demo` 的資料集來存放資料表。

1. 在本程式碼研究室中，我們將執行 SQL 指令。您可以在 **BigQuery Studio > SQL 編輯器** 中執行這些指令，也可以在 Cloud Shell 中使用 `bq query` 指令。



我們假設您使用 **BigQuery SQL 編輯器**，以便更順暢地使用多行建立陳述式。

```
CREATE SCHEMA IF NOT EXISTS `fraud_demo` OPTIONS(location="US");
```

載入資料表

執行下列 SQL 陳述式，將資料從 Cloud Storage 載入資料集。

```
LOAD DATA OVERWRITE `fraud_demo.customers`
FROM FILES (
  format = 'CSV',
  uris = ['gs://sample-data-and-media/fraud-demo-data/customers.csv'],
  skip_leading_rows = 1
);

LOAD DATA OVERWRITE `fraud_demo.emails`
FROM FILES (
  format = 'CSV',
  uris = ['gs://sample-data-and-media/fraud-demo-data/emails.csv'],
  skip_leading_rows = 1
);

LOAD DATA OVERWRITE `fraud_demo.phones`
FROM FILES (
  format = 'CSV',
  uris = ['gs://sample-data-and-media/fraud-demo-data/phones.csv'],
  skip_leading_rows = 1
);

LOAD DATA OVERWRITE `fraud_demo.addresses`
FROM FILES (
  format = 'CSV',
  uris = ['gs://sample-data-and-media/fraud-demo-data/addresses.csv'],
  skip_leading_rows = 1
);

LOAD DATA OVERWRITE `fraud_demo.customer_emails`
FROM FILES (
  format = 'CSV',
  uris = ['gs://sample-data-and-media/fraud-demo-data/customer_emails.csv'],
  skip_leading_rows = 1
);

LOAD DATA OVERWRITE `fraud_demo.customer_phones`
FROM FILES (
  format = 'CSV',
  uris = ['gs://sample-data-and-media/fraud-demo-data/customer_phones.csv'],
  skip_leading_rows = 1
);

LOAD DATA OVERWRITE `fraud_demo.customer_addresses`
FROM FILES (
  format = 'CSV',
  uris = ['gs://sample-data-and-media/fraud-demo-data/customer_addresses.csv'],
  skip_leading_rows = 1
);

LOAD DATA OVERWRITE `fraud_demo.orders`
FROM FILES (
  format = 'CSV',
```

```
uris = ['gs://sample-data-and-media/fraud-demo-data/orders.csv'],  
skip_leading_rows = 1  
);
```

4. 建立屬性圖表

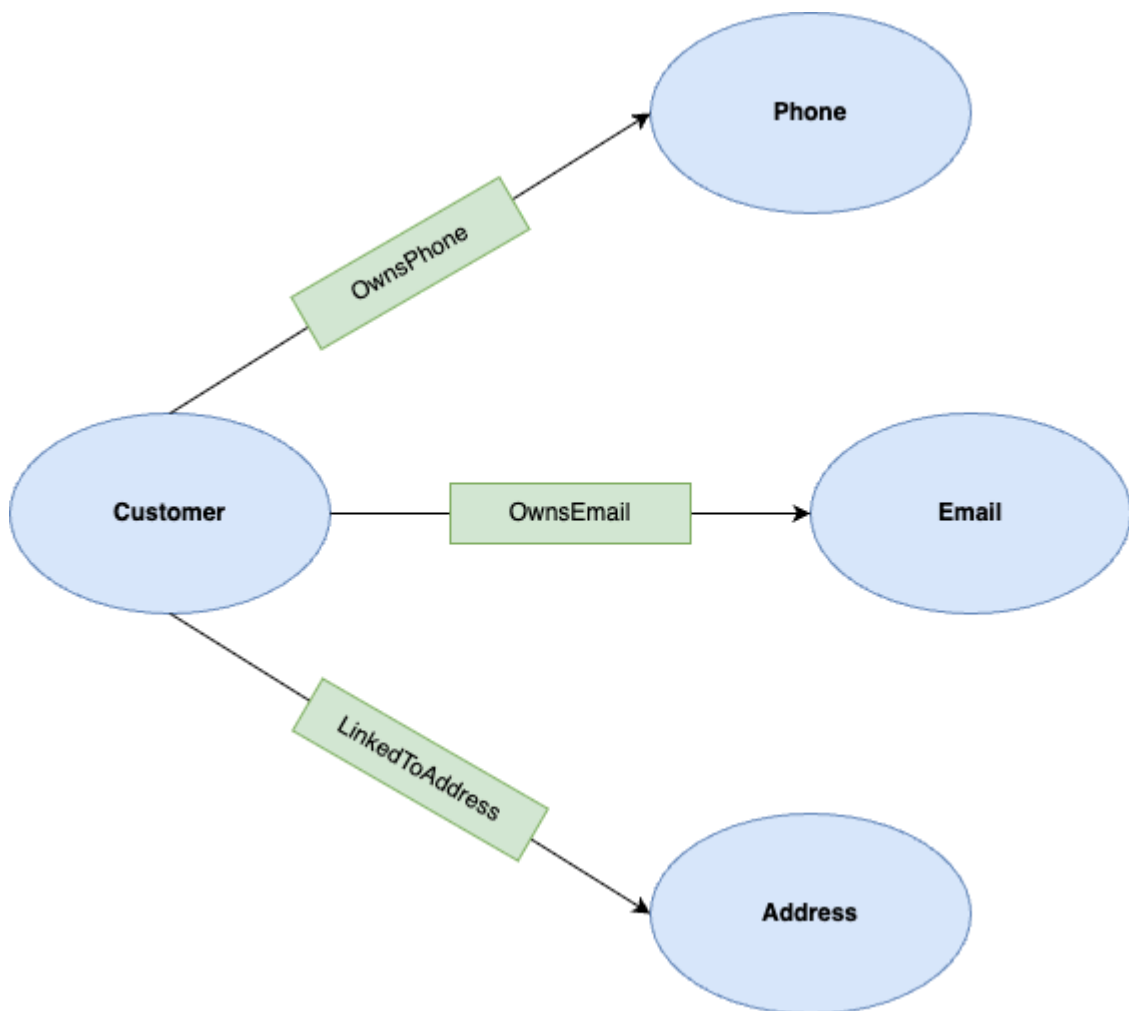
資料載入完畢後，您就可以定義屬性圖。屬性圖形由節點 (實體) 和邊緣 (關係) 組成。

在本實驗室中，節點如下：

- **客戶**：代表帳戶持有人。
- **電話**：代表電話號碼。
- **電子郵件**：代表電子郵件地址。
- **地址**：代表實際地址。

邊緣為：

- **OwnsPhone**：將顧客連結至電話。
- **OwnsEmail**：將顧客連結至電子郵件。
- **LinkedToAddress**：將客戶連結至地址。



建立圖表

執行下列 DDL 陳述式，在 `fraud_demo` 資料集中建立名為 `FraudDemo` 的圖表。

```

CREATE OR REPLACE PROPERTY GRAPH fraud_demo.FraudDemo
  NODE TABLES(
    fraud_demo.customers
      KEY(account_id)
      LABEL Customer PROPERTIES(
        account_id,
        name),

    fraud_demo.emails
      KEY(email)
      LABEL Email PROPERTIES(
        email,
        email_type),

    fraud_demo.phones
      KEY(phone_number)
      LABEL Phone PROPERTIES(
        phone_number,
        phone_type),

    fraud_demo.addresses
      KEY(address)
      LABEL Address PROPERTIES(
        address,
        address_type)
  )
  EDGE TABLES(
    fraud_demo.customer_emails
      KEY(account_id, email)
      SOURCE KEY(account_id) REFERENCES customers(account_id)
      DESTINATION KEY(email) REFERENCES emails(email)
      LABEL OwnsEmail PROPERTIES(
        account_id,
        email,
        last_updated_ts),

    fraud_demo.customer_phones
      KEY(account_id, phone_number)
      SOURCE KEY(account_id) REFERENCES customers(account_id)
      DESTINATION KEY(phone_number) REFERENCES phones(phone_number)
      LABEL OwnsPhone PROPERTIES(
        account_id,
        phone_number,
        last_updated_ts),

    fraud_demo.customer_addresses
      KEY(account_id, address)
      SOURCE KEY(account_id) REFERENCES customers(account_id)
      DESTINATION KEY(address) REFERENCES addresses(address)
      LABEL LinkedToAddress PROPERTIES(
        account_id,
        address,

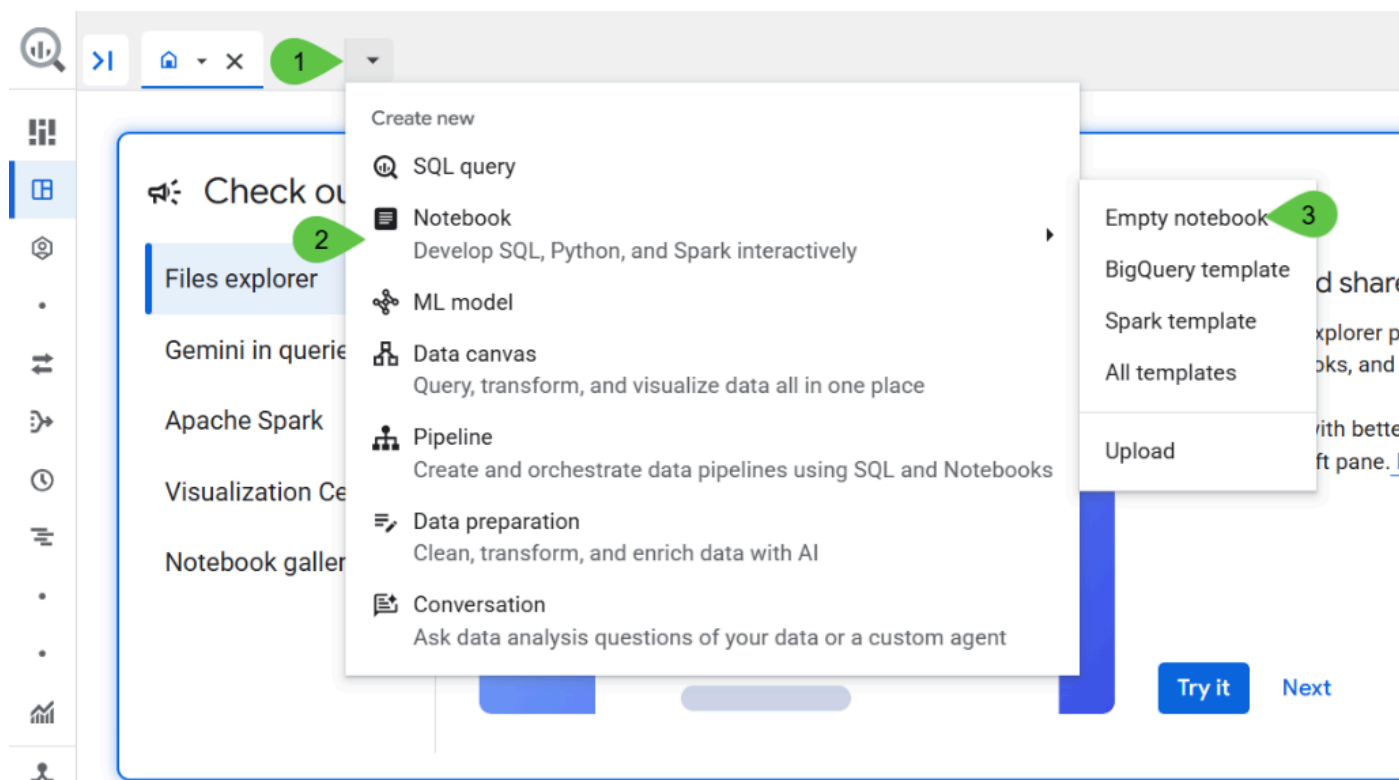
```

```
last_updated_ts)  
);
```

步驟 5 · 約 5 分鐘

5. 分析網路 (2 個躍點)

在 BigQuery Studio 中開啟「新筆記本」。



在本程式碼實驗室的視覺化和建議部分，我們將使用 BigQuery Studio 中的 **Google Colab** 筆記本。這樣我們就能輕鬆查看圖表結果。

將下列內容貼到程式碼儲存格：

```
!pip install bigquery-magics==0.12.1
```

BigQuery Graph Notebook 會以 IPython Magics 實作。新增 `%bigquery` 魔法指令和 `TO_JSON` 函式後，即可如後續章節所示，將結果視覺化。在這個步驟中，您將執行圖形查詢，找出帳戶之間的簡單連結。這是「2 跳」查詢，因為查詢會從起始節點跳出 2 次，尋找相關節點 (例如：客戶 -> 電子郵件 -> 客戶)。

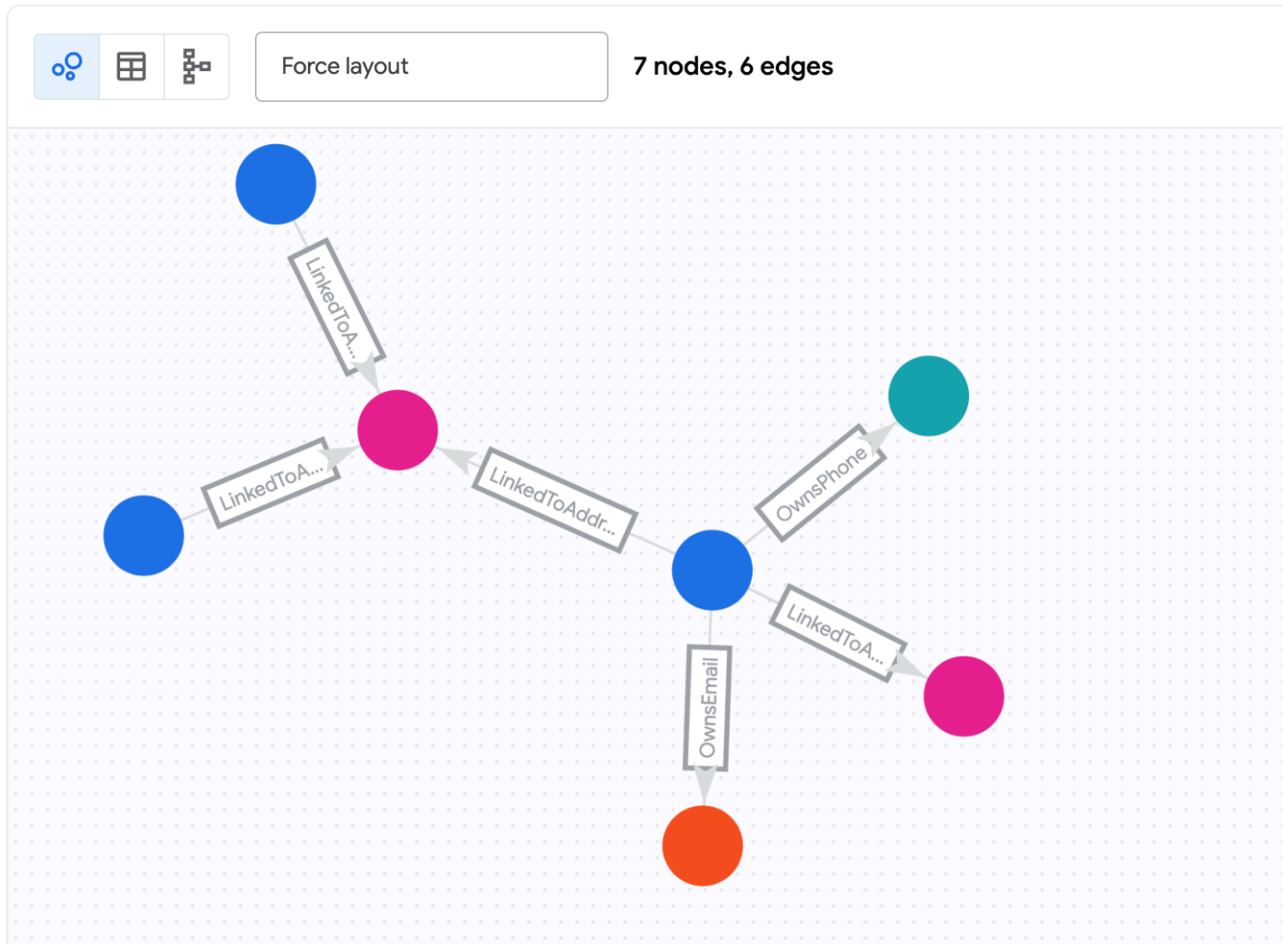
我們將先調查 **Nicole Wade** 的帳戶。我們想透過 2 個跳轉次數，找出與她相關的帳戶。

執行 2-Hop 查詢

在筆記本中執行下列查詢。

```
%%bigquery --graph
GRAPH fraud_demo.FraudDemo
MATCH
  p=(a:Customer)
  ( -[e:OwnsEmail|OwnsPhone|LinkedToAddress WHERE e.last_updated_ts < '2025-07-30']- (n) ){2}
WHERE a.account_id IN ("d2f1f992-d116-41b3-955b-6c76a3352657")
  -- Verify the final node in the hop array is a Customer
  AND 'Customer' IN UNNEST(LABELS(n[OFFSET(1)]))

RETURN TO_JSON(p) AS paths
```



瞭解結果

這項查詢會進行下列操作：

1. 從 `Customer` 節點開始，節點為「d2f1f992-d116-41b3-955b-6c76a3352657」(Nicole Wade)。`account_id`
2. 沿著任一邊緣 (`OwnsEmail`、`OwnsPhone` 或 `LinkedToAddress`) 前往連接節點 (`Phone`、`Email` 或 `Address`)。
3. 從該連結節點沿著邊緣返回其他 `Customer` 節點。
4. 根據時間戳記 (`last_updated_ts`) 篩選邊緣，查看特定時間的網路狀態。

您應該會看到 **Zachary Cordova** 和 **Brenda Brown** 透過相同地址與 Nicole 連結。

6. 分析網路 (4 個躍點)

在這個步驟中，您將擴充查詢，找出更複雜的關係。我們會尋找 **4 跳**連線。這讓我們能夠找出透過多個中介實體連結的帳戶 (例如：客戶 A -> 電子郵件 -> 客戶 B -> 電話 -> 客戶 C)。

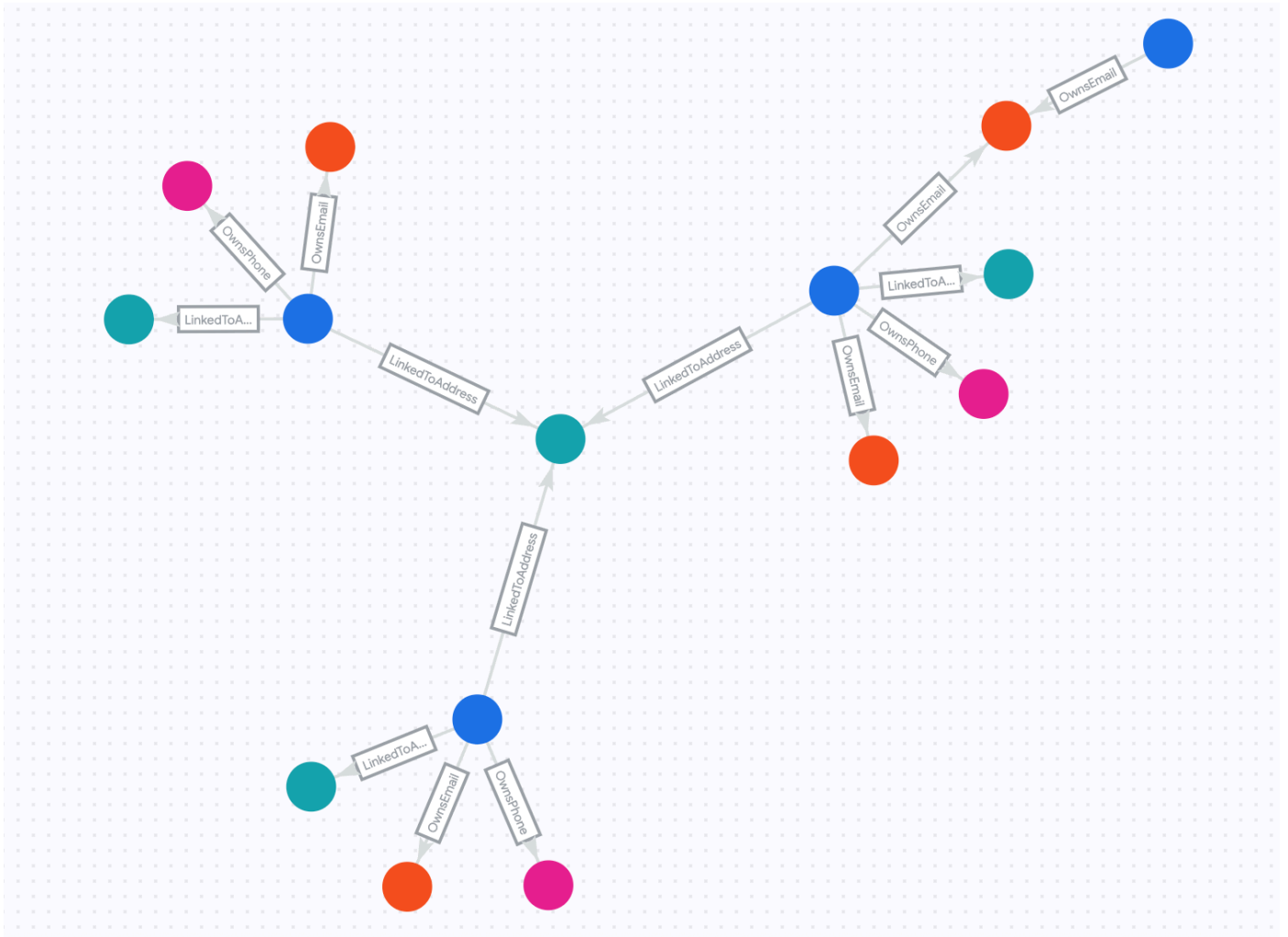
我們也會觀察這個網路隨時間的變化。

「之前」狀態

首先，我們來看看 **2025 年 7 月 30 日**的電視網。

執行以下查詢：

```
%%bigquery --graph
GRAPH fraud_demo.FraudDemo
MATCH p= ANY SHORTEST (a:Customer)
    ( -[e:OwnsEmail|OwnsPhone|LinkedToAddress WHERE e.last_updated_ts < '2025-07-30']- (n) ){3,
5}(reachable_a:Customer)
WHERE a.account_id IN ("d2f1f992-d116-41b3-955b-6c76a3352657")
    -- Ensure the final node in the dynamic chain is actually a Customer
    AND 'Customer' IN UNNEST(LABELS(n[OFFSET(ARRAY_LENGTH(n) - 1)]))
RETURN
    TO_JSON(p) AS paths,                -- Array of all traversed edges
    ARRAY_LENGTH(e) AS hop_count
```



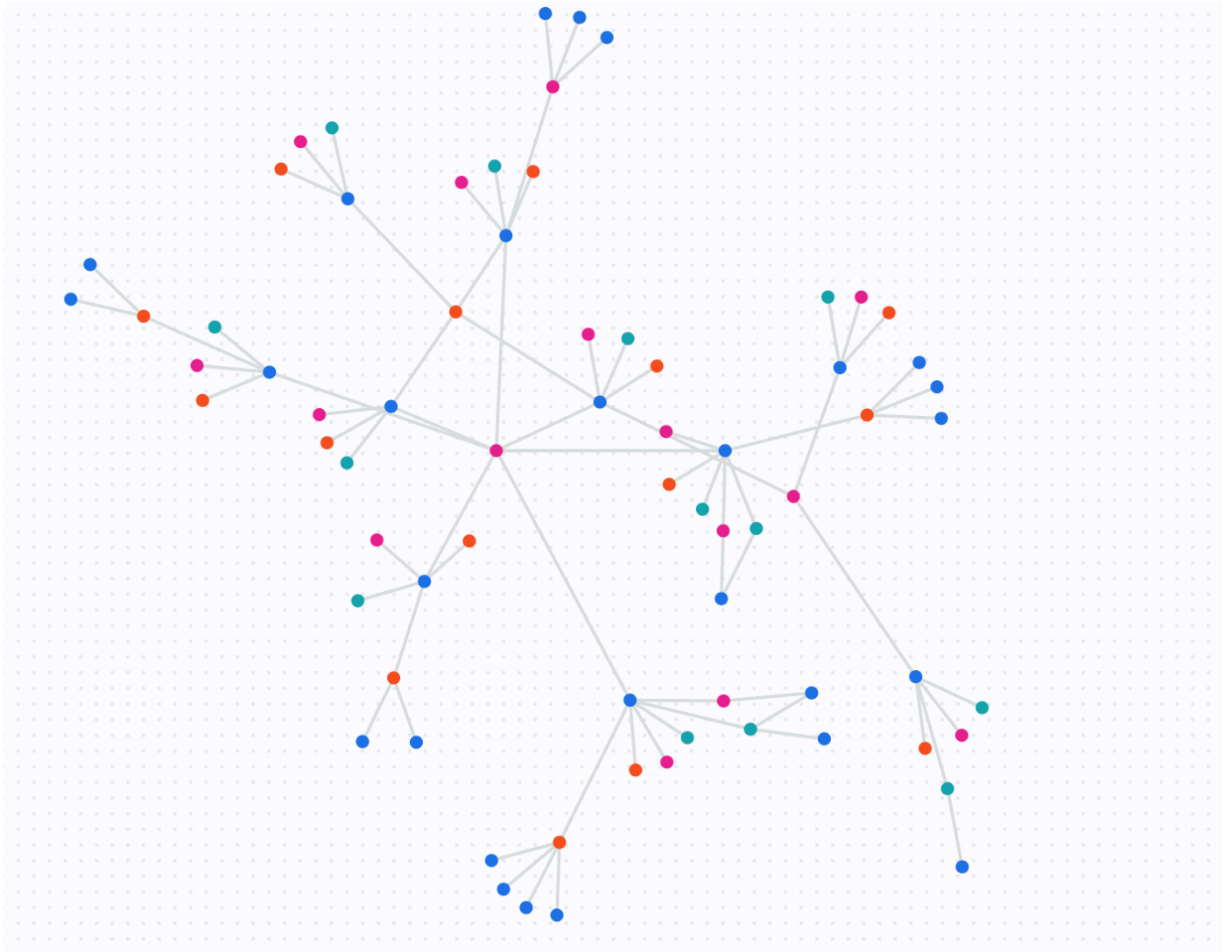
「之後」狀態

現在，讓我們看看 **2 週後** 的網路。我們會執行相同的查詢，但不會設下日期限制。

執行以下查詢：

```
%%bigquery --graph
GRAPH fraud_demo.FraudDemo
MATCH p= ANY SHORTEST (a:Customer)
  ( -[e:OwnsEmail|OwnsPhone|LinkedToAddress]- (n) ){3, 5}(reachable_a:Customer)
WHERE a.account_id IN ("d2f1f992-d116-41b3-955b-6c76a3352657")
  -- Ensure the final node in the dynamic chain is actually a Customer
  AND 'Customer' IN UNNEST(LABELS(n[OFFSET(ARRAY_LENGTH(n) - 1)]))

RETURN
  TO_JSON(p) AS paths,                -- Array of all traversed edges
  ARRAY_LENGTH(e) AS hop_count
```



瞭解結果

移除日期篩選器後，系統會對整個資料集執行查詢。你會發現網路大幅擴展。**Nicole Wade** 現在是規模更大的高連結群組成員。如果連線網路快速擴展，很可能代表有詐欺活動，例如詐欺集團長期共用資源。

步驟 7 · 約 15 分鐘

7. 產生詐欺報表

在這個步驟中，您會將圖表分析與傳統業務資料 (訂單) 結合，產生完整的詐欺報表。找出有風險的帳戶和潛在的詐欺訂單。

這項查詢較為複雜，這個函式會使用 `GRAPH_TABLE` 在標準 SQL 中執行圖表查詢，並計算我們在上一個步驟觀察到的「之前」和「之後」狀態之間的網路大小變化 (`diff`)。

執行詐欺報表查詢

在筆記本中執行下列查詢。

```

%%bigquery --graph
WITH num_orders AS (
  SELECT account_id, COUNT(1) AS num_order
  FROM fraud_demo.orders
  WHERE order_time > '2025-07-30'
  GROUP BY account_id
),

orders AS (
  SELECT account_id, order_id, fraud, order_total
  FROM fraud_demo.orders
  WHERE order_time > '2025-07-30'
),

-- Use Quantified Path Patterns to find connections up to 4 hops away
latest_connect AS (
  SELECT
    account_id,
    ARRAY_LENGTH(ARRAY_AGG(DISTINCT connected_id)) AS size
  FROM GRAPH_TABLE(
    fraud_demo.FraudDemo
    MATCH (a:Customer)-[:OwnsEmail|OwnsPhone|LinkedToAddress]-{4}(connected:Customer)
    RETURN a.account_id AS account_id, connected.account_id AS connected_id
  )
  GROUP BY account_id
),

prev_connect AS (
  SELECT
    account_id,
    ARRAY_LENGTH(ARRAY_AGG(DISTINCT connected_id)) AS size
  FROM GRAPH_TABLE(
    fraud_demo.FraudDemo
    -- Apply the timestamp filter to EVERY edge in the 4-hop chain
    MATCH (a:Customer)
      (-[e:OwnsEmail|OwnsPhone|LinkedToAddress WHERE e.last_updated_ts < '2025-07-30']-
(n)){4}
    WHERE 'Customer' IN UNNEST(LABELS(n[OFFSET(3)]))
    RETURN a.account_id AS account_id, n[OFFSET(3)].account_id AS connected_id
  )
  GROUP BY account_id
),

edge_changes AS (
  SELECT account_id, MAX(last_updated_ts) AS max_last_updated_ts
  FROM fraud_demo.customer_addresses
  GROUP BY account_id
)

SELECT
  la.account_id,
  o.order_id,

```

```

    la.size AS latest_size,
    COALESCE(pa.size, 0) AS previous_size,
    la.size - COALESCE(pa.size, 0) AS diff,
    nos.num_order,
    o.fraud AS reported_as_fraud,
    o.order_total,

    CASE
        WHEN (la.size - COALESCE(pa.size, 0)) > 10 AND nos.num_order IS NULL THEN "CUSTOMER AT RISK"
        WHEN (la.size - COALESCE(pa.size, 0)) > 10 AND nos.num_order IS NOT NULL AND o.fraud THEN "CONFIRMED FRAUD ORDER"
        WHEN (la.size - COALESCE(pa.size, 0)) > 10 AND nos.num_order IS NOT NULL AND NOT o.fraud THEN "POTENTIAL FRAUD ORDER"
        ELSE ""
    END AS notes
FROM latest_connect la
LEFT JOIN prev_connect pa ON la.account_id = pa.account_id
LEFT JOIN num_orders nos ON la.account_id = nos.account_id
LEFT JOIN orders o ON la.account_id = o.account_id
INNER JOIN edge_changes ec ON la.account_id = ec.account_id
WHERE nos.num_order > 1 OR (la.size - COALESCE(pa.size, 0)) > 10
ORDER BY diff DESC

```

瞭解結果

這份報表會顯示：

- `account_id`：要分析的帳戶 ID。
- `order_id`：最近的訂單 ID。
- `latest_size`：今天連線網路的大小。
- `previous_size`：2 週前的網路大小。
- `diff`：網路規模的成長。
- `num_order`：近期訂單數量。
- `reported_as_fraud`：訂單是否已標記為詐欺。
- `order_total`：訂單總金額。
- `notes`：根據網路成長和訂單記錄計算出的風險狀態。

您會看到 `diff` 值較大且訂單總計金額較高的帳戶，這些帳戶是進一步調查的首要對象。「CUSTOMER AT RISK」(有流失風險的顧客) 和「POTENTIAL FRAUD ORDER」(可能為詐欺訂單) 附註有助於優先處理這些帳戶。

index	account_id	order_id	latest_size	previous_size	diff	num_order	reported_as_fraud	order_total	notes
0	219ab4e6-dcf1-4d2c-bc7a-d9f993bbff9f		40	2	38	<NA>	<NA>	NaN	CUSTOMER AT RISK
1	94087590-c9b6-42d4-a5d4-21182b669ac7		38	3	35	<NA>	<NA>	NaN	CUSTOMER AT RISK
2	3ff59684-fbf9-40d7-8c41-285ade5002e6	46a36a95-5961-41f0-99f0-f3dc5bd79e49	34	1	33	1	true	931.05	CONFIRMED FRAUD ORDER
3	bca72488-bfbe-4e19-9379-fbcedfc14b45		33	2	31	<NA>	<NA>	NaN	CUSTOMER AT RISK
4	66e5946b-796c-4d34-ad87-a32aea5861da		30	4	26	<NA>	<NA>	NaN	CUSTOMER AT RISK
5	9fe66691-6f3d-47b6-b803-00e7f6d59c52		27	1	26	<NA>	<NA>	NaN	CUSTOMER AT RISK
6	392fe0af-dc75-4134-a364-e4909cfca1a1		27	3	24	<NA>	<NA>	NaN	CUSTOMER AT RISK
7	b5a42ef2-8674-4ef3-ab26-3fdc7d7812e8		27	3	24	<NA>	<NA>	NaN	CUSTOMER AT RISK
8	845f2b14-cd10-4750-9f28-fe542c4a731b	48cdfc76-b892-438d-9ea8-14d62d23c255	25	1	24	2	true	659.52	CONFIRMED FRAUD ORDER
9	845f2b14-cd10-4750-9f28-fe542c4a731b	c372db55-82af-493b-a653-1720ef912904	25	1	24	2	true	485.88	CONFIRMED FRAUD ORDER
10	3f24a037-14c3-4a70-af01-156073e6a8b8		28	4	24	<NA>	<NA>	NaN	CUSTOMER AT RISK

步驟 8 · 約 5 分鐘

8. 大規模偵測

在最後的分析步驟中，您將以較大的規模呈現網路。您會查詢一組可疑帳戶之間的關聯，而不是從單一帳戶開始。有助您判斷多項獨立調查是否其實屬於同一個大型詐欺集團。

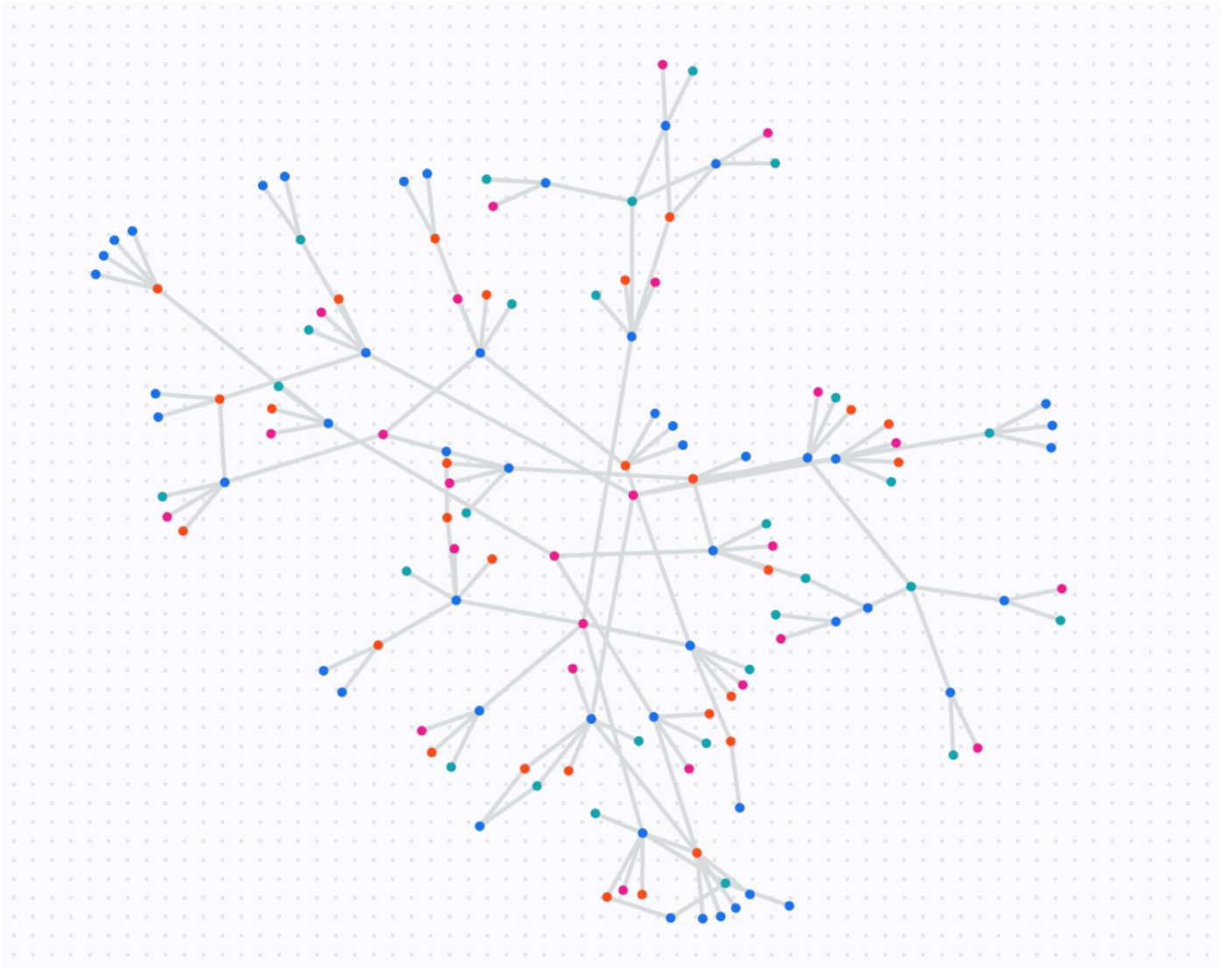
執行縮放查詢

在筆記本中執行下列查詢。

```
%%bigquery --graph
GRAPH fraud_demo.FraudDemo
MATCH
p= ANY SHORTEST (a:Customer)
  ( -[e:OwnsEmail|OwnsPhone|LinkedToAddress]- (n) ){3, 5}
(reachable_a:Customer)
-- these IDs are from the previous results
WHERE a.account_id in (  "845f2b14-cd10-4750-9f28-fe542c4a731b"
  , "3ff59684-fbf9-40d7-8c41-285ade5002e6"
  , "8887c17b-e6fb-4b3b-8c62-cb721aafd028"
  , "03e777e5-6fb4-445d-b48c-cf42b7620874"
  , "81629832-eb1d-4a0e-86da-81a198604898"
  , "845f2b14-cd10-4750-9f28-fe542c4a731b",
  "89e9a8fe-ffc4-44eb-8693-a711a3534849"
)
LIMIT 400
RETURN TO_JSON(p) as paths
```

瞭解結果

這項查詢會傳回複雜的圖表，顯示指定可疑帳戶的重疊情形和共用資源。您現在正在大規模偵測詐欺行為，找出可能需要協調回應的活動叢集。



步驟 9 · 約 2 分鐘

9. 清除

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請刪除資料集和屬性圖。

執行下列 SQL 陳述式，清理環境。

```
DROP PROPERTY GRAPH IF EXISTS fraud_demo.FraudDemo;  
DROP SCHEMA IF EXISTS fraud_demo CASCADE;
```

步驟 10 · 約 0 分鐘

10. 恭喜

恭喜！您已成功使用 BigQuery 圖表建構詐欺偵測解決方案。

您已學會以下技能：

- 將資料從 Cloud Storage 載入 BigQuery。
- 使用 DDL 定義屬性圖表。
- 使用 GQL 查詢圖形，找出簡單和複雜的關係。
- 結合圖表分析與業務資料，找出風險。
- 大規模呈現網路。

其他資源

- [BigQuery Graph 說明文件](#)
-